Laboratory Exercise 3
Arithmetic Circuits and Introduction to VHDL

Introduction

This laboratory exercise accompanies Lecture 4 (Number Representation and Arithmetic Circuits). You will build a full adder from gates on the breadboard, and then describe the same circuit in **VHDL**, the hardware description language used from this point to the end of the course. The full adder is then used as a building block of a 4-bit ripple-carry adder and an adder/subtractor for two's complement numbers with overflow detection. The VHDL part of this laboratory is a guided introduction: all the VHDL syntax that you need is explained here, and the code you must type is mostly given.

Objectives

1. To build and test a full adder from XOR, AND and OR gates.
2. To write, compile and simulate a first VHDL design (entity, architecture, signals, concurrent assignments).
3. To build a larger circuit from smaller ones by component instantiation (structural VHDL).
4. To implement a 4-bit ripple-carry adder (and, as an extension, a two's complement adder/subtractor with overflow detection).
5. (Extension) To observe that the delay of a ripple-carry adder grows with the number of bits.

Equipment Required

- DC Power Supply (+5V), Digital Multimeter
- Breadboard and Jumper Wires
- TTL ICs: 74LS86 (XOR), 74LS08 (AND), 74LS32 (OR), 74LS04 (NOT, for LED indicator stages)
- Resistors: 330Ω, DIP switches, LEDs
- Intel DE0-CV board and a PC with Quartus Prime

Pre-lab (complete before the session)

The session is two hours long. Read the subsection "Anatomy of a VHDL file" of Part II *before* the session, and complete:

1. Convert the following to 4-bit two's complement and fill Table 1: $+5$, -3, -8. Compute $5+(-3)$, $5+6$ and $-5+(-4)$ in 4-bit two's complement and state whether each operation **overflows**.
2. Write the Boolean equations of a full adder: $S =$ _____ $C_{out} =$ _____

Operation	A (4-bit)	B (4-bit)	Result (4-bit)	Overflow?
$5 + (-3)$				
$5 + 6$				
$(-5) + (-4)$				

Table 1: Pre-lab: two's complement arithmetic.

Part I: Full Adder from Gates (about 25 min)

A full adder adds three bits A, B and the carry-in C_{in} and produces a sum S and a carry-out C_{out}. As shown in Lecture 4, it can be built from two half adders and an OR gate (Figure 1).

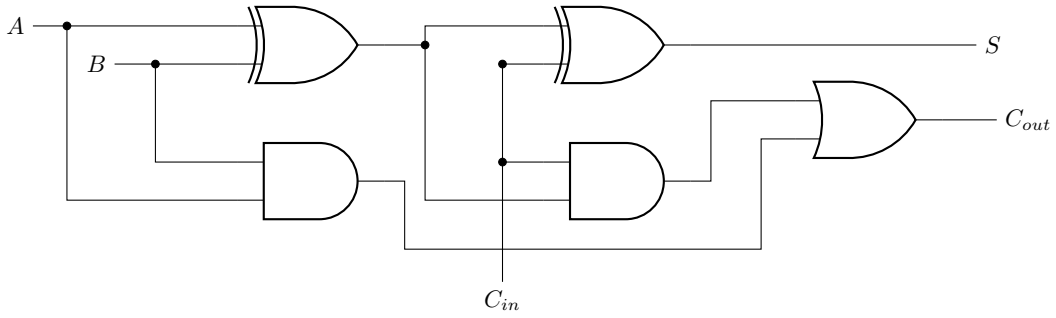


Figure 1: Full adder built from two half adders (XOR + AND) and an OR gate. Wires that cross without a dot are not connected.

1. Using 74LS86 (2 XOR gates), 74LS08 (2 AND gates) and 74LS32 (1 OR gate), build the full adder on the breadboard. Redraw a clean schematic in your report with the pin numbers.
2. Connect A, B, C_{in} to three switches, and S and C_{out} to LED indicator stages. (Tip: draw and test the circuit first in Logisim, so you can find wiring mistakes before touching the breadboard.)
3. Complete Table 2. Does the output satisfy $A + B + C_{in} = 2\,C_{out} + S$ for each row?

A	B	C_{in}	S (measured)	C_{out} (measured)	S (VHDL sim.)	C_{out} (VHDL sim.)
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

Table 2: Full-adder truth table (the last two columns are completed in Part II).

Part II: Your First VHDL Design (about 30 min)

Anatomy of a VHDL file

A VHDL file describes a circuit in two parts (see Figure 2):

- The **entity** lists the interface of the circuit: its input and output ports (like the pins of an IC).
- The **architecture** describes what happens inside. The symbol `<=` is a *signal assignment*. Statements in an architecture are **concurrent**: they all describe hardware that operates at the same time, so the order of the statements does not matter.

The logic operators are `NOT`, `AND`, `OR`, `XOR` and `NAND`, `NOR`, `XNOR`. A `SIGNAL` declared in the architecture is like an internal wire.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY full_adder IS
    PORT ( A, B, Cin : IN  STD_LOGIC;
           S, Cout   : OUT STD_LOGIC );
END full_adder;

ARCHITECTURE dataflow OF full_adder IS
    SIGNAL x : STD_LOGIC;           -- internal wire: A XOR B
BEGIN
    x    <= A XOR B;
    S    <= x XOR Cin;
    Cout <= (A AND B) OR (x AND Cin);
END dataflow;
```

Figure 2: VHDL description of a full adder.

1. Create a new Quartus project `lab3` for the DE0-CV board and a new VHDL file named `full_adder.vhd`. The file name must be the same as the entity name. Type the code of Figure 2.
2. Compile the file. How many logic elements (or ALMs) are used? _____
3. Simulate the design. Create a waveform with A, B, C_{in} counting through all 8 combinations, and compare the result with the third and fourth column of Table 2. Fill in the last two columns of this table.
4. Open the **RTL Viewer** (**Tools** → **Netlist Viewers** → **RTL Viewer**). Does it resemble Figure 1?
5. Fill the pin assignments to use SW0, SW1, SW2 as A, B, C_{in} and LEDR0, LEDR1 as S, C_{out}, then download to the board and test it. Check that the board behaves like your breadboard circuit.

Part III: 4-bit Ripple-Carry Adder (about 40 min)

A 4-bit adder is made by connecting four full adders so that each carry-out feeds the carry-in of the next stage (Figure 3). In VHDL, an existing entity is used as a **component**: it is declared in the architecture and instantiated with a `PORT MAP`, which connects each port of the component to a signal.

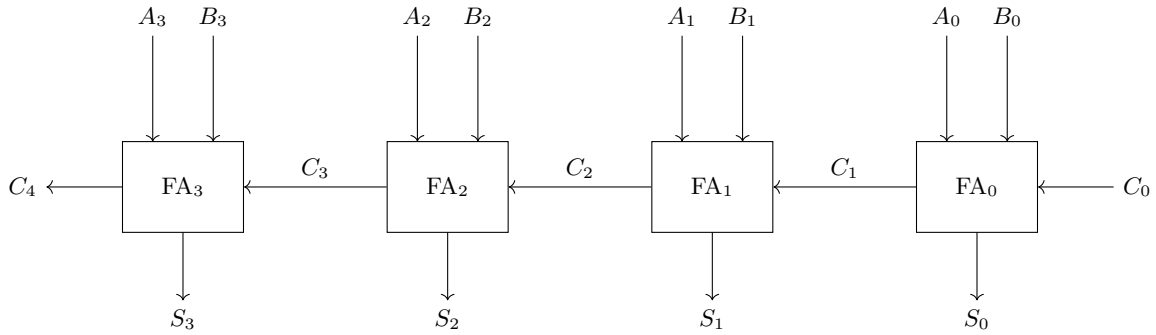


Figure 3: 4-bit ripple-carry adder.

```vhdl
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY ripple_adder_4bit IS
    PORT ( A, B : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
           Cin  : IN  STD_LOGIC;
           S    : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
           Cout : OUT STD_LOGIC );
END ripple_adder_4bit;

ARCHITECTURE structural OF ripple_adder_4bit IS
    COMPONENT full_adder
        PORT ( A, B, Cin : IN  STD_LOGIC;
               S, Cout   : OUT STD_LOGIC );
    END COMPONENT;
    SIGNAL C : STD_LOGIC_VECTOR(3 DOWNTO 1);   -- internal carries C1..C3
BEGIN
    FA0: full_adder PORT MAP ( A => A(0), B => B(0), Cin => Cin,  S => S(0), Cout => C(1) );
    FA1: full_adder PORT MAP ( A => A(1), B => B(1), Cin => C(1), S => S(1), Cout => C(2) );
    -- TODO: instantiate FA2 and FA3 in the same way.
    -- FA3 must produce the carry-out of the whole adder (Cout).
END structural;
```

Figure 4: Skeleton of the structural description of the 4-bit ripple adder.

1. Add `ripple_adder_4bit.vhd` to the project, complete the two missing instances, set it as the top-level entity and compile it. A `STD_LOGIC_VECTOR(3 DOWNTO 0)` is a bundle of four wires, indexed from 3 (MSB) down to 0 (LSB). Number of logic elements used: _____

2. Simulate the adder. Apply at least the test cases in Table 3 and check that the results are correct for *unsigned* numbers.

3. Assign A to SW3–SW0, B to SW7–SW4, C_{in} to SW8, S to LEDR3–LEDR0 and C_{out} to LEDR4. Download to the board and verify the test cases.

A (hex)	B (hex)	C_{in}	**Expected** $A + B + C_{in}$ (decimal)	C_{out}	S (hex)	Match?
3	4	0				
7	8	0				
F	1	0				
F	F	1				

Table 3: Test cases for the 4-bit ripple-carry adder (unsigned).

Extension: Adder/Subtractor and Overflow Detection (take-home, or if time permits)

Propagation Delay of the Ripple Adder

1. Run the **Timing Analyzer** (**Tools** → **Timing Analyzer**, use **Report Timing**) and find the longest propagation delay from an input to an output of your 4-bit adder: $t_{pd,\text{4-bit}}$ = _____ ns.

2. Extend the design to **8 bits** by instantiating two copies of `ripple_adder_4bit` (Lecture 4, Exercise 4-4) and find the longest delay again: $t_{pd,\text{8-bit}}$ = _____ ns. How does the delay scale with the number of bits? Why does a Carry Lookahead Adder avoid this?

Adder/Subtractor

Using two's complement numbers, $A - B = A + \overline{B} + 1$. An adder can be turned into an adder/subtractor by passing B through XOR gates controlled by a mode input M ($M = 0$: add, $M = 1$: subtract) and also connecting M to the carry-in (Figure 5). Overflow of signed numbers occurs when the carry into the MSB differs from the carry out of the MSB: $V = C_3 \oplus C_4$.

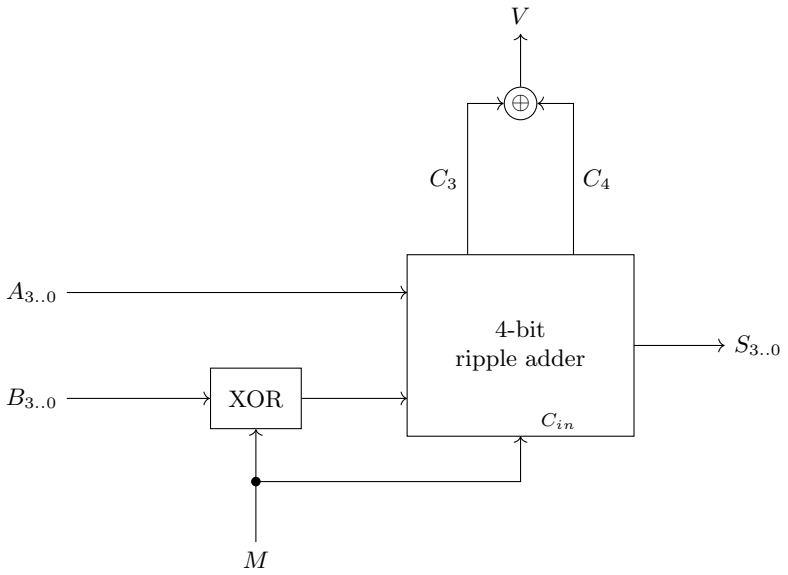


Figure 5: Adder/subtractor. M is also connected to C_{in} of the adder. $V = C_3 \oplus C_4$.

1. Modify `ripple_adder_4bit` (or write a new entity `addsub_4bit`) so that the internal carry C_3 is available. Hint: add an output port `C3 : OUT STD_LOGIC` and write `C3 <= C(3);`, then write the top-level entity with `Bx <= B XOR (M & M & M & M);` where `&` is the concatenation operator.

2. Add the overflow output V and the mode input M to your top-level. Assign A to SW3–SW0, B to SW7–SW4, M to SW9, S to LEDR3–LEDR0 and V to LEDR9.

3. Simulate and test on the board the cases in Table 4. Interpret A, B and S as **signed** 4-bit numbers and give the signed decimal value of the result.

4. The subtraction $A - B$ with $A = 0000_2$ and $B = 1000_2$ (-8) is a special case. What does the circuit output, and why is it wrong?

M	A	B	A (dec)	B (dec)	S (bin)	S (dec)	V
0	0101	0011					
0	0101	0110					
0	1011	1100					
1	0101	0011					
1	0011	0101					
1	0111	1110					

Table 4: Test cases for the signed adder/subtractor.

Check-off

Show your instructor or teaching assistant:

1. The breadboard full adder (Table 2) and the same circuit in VHDL on the board.

2. The 4-bit ripple adder in simulation and on the board (Table 3).

Updated By: R. Sutthaweekul
Version: 1.0
Release Date: 2026-11-11